

Two Pointers & Sliding Window

This chapter practices a family of techniques for **optimizing brute-force nested loops**: many "find a subarray/substring" problems have an obvious $O(n^2)$ solution using two nested for loops, and two-pointer / sliding-window techniques bring that down to $O(n)$. The core skill is judging which pointer-movement pattern a problem calls for.

- Python 3
- Tutor style
- Includes diagrams
- Includes FAQ
- 4 pointer patterns

00 Judgment Rule + Review Order

First figure out which pointer-movement pattern the problem calls for — the answer tells you exactly which template to reach for.

WHAT THE PROBLEM LOOKS LIKE	WHICH PATTERN
Sorted array, find a pair satisfying some condition	Opposite pointers (left/right close in from both ends)
Check whether a string reads the same forwards/backwards	Opposite pointers (palindrome check)
In-place dedupe/filter, no extra space allowed	Fast/slow pointers (same direction)
Subarray/substring length is fixed at k	Fixed-size sliding window
Length isn't fixed, find the shortest/longest	Variable-size sliding window (expand right, shrink left)

- 01 ← start

Opposite pointers

Two-sum sorted
- 02

Palindrome

Same pattern, new use
- 03

Fast/slow

In-place dedupe
- 04

Fixed window

Slide, don't re-sum
- 05

Shrinking window

Shortest valid
- 06

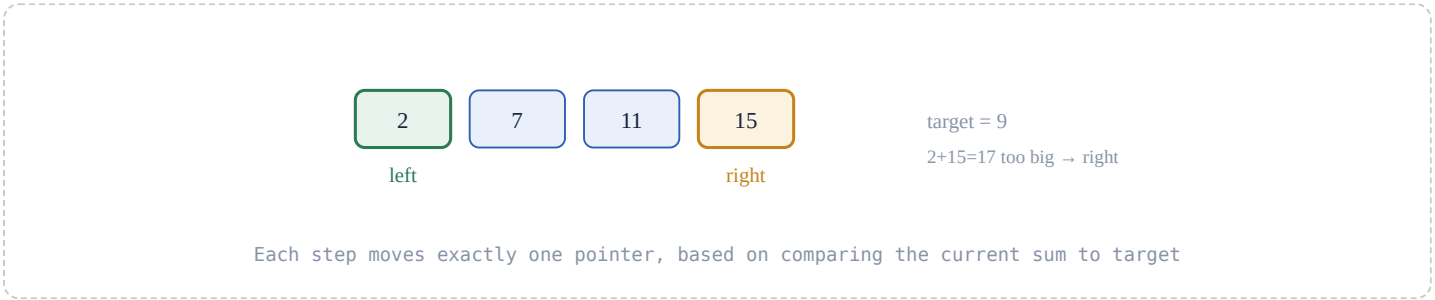
Dedup window

Longest valid

The one-sentence test: "Can this problem be solved with a single pass, using two pointers each walking one side, instead of the nested loops it would otherwise need?" If yes, you're in this chapter's territory — the only remaining question is which of the four movement patterns fits.

01 Opposite Pointers: Exploiting "Already Sorted"

The core prerequisite: **the array must already be sorted**, or the logic "sum too small, so move left" doesn't hold.



two_sum_sorted — one pass, $O(n)$

opposite

```
left = 0
right = len(nums) - 1
while left < right:
    total = nums[left] + nums[right]
    if total == target:
        return [left, right]
    elif total < target:
        left += 1    # sum too small — only moving left can make it bigger
    else:
        right -= 1   # sum too big — only moving right can make it smaller
```

The critical prerequisite: the array must be sorted. Otherwise "sum too small so move left right" doesn't hold — without sorting, moving left doesn't guarantee the sum will actually increase. One pointer starts at the far left, the other at the far right; each step compares the current sum to target and moves exactly one pointer. Across the whole run, **left** / **right** together move at most n steps total, giving $O(n)$ — far better than the $O(n^2)$ of checking every pair.

Use case: sorted arrays, "pair of numbers satisfying some condition"

02 Opposite Pointers, Another Use: Palindrome Check

is_palindrome — same movement, different comparison

palindrome

```
left = 0
right = len(text) - 1
while left < right:
    if text[left] != text[right]:
        return False
    left += 1
    right -= 1
return True
```

This is structurally the exact same pointer movement as § 01 (both ends closing toward the middle) — the only difference is what gets compared: instead of "does the sum equal target," it's "are the two end characters equal." This proves **opposite pointers aren't limited to numeric sums** — any problem shaped like "check both ends simultaneously, converge toward the middle" fits this pattern.

Use case: checking left-right symmetry in a string/array, or any problem needing to check corresponding front/back positions at once

03 Fast / Slow Pointers (Same-Direction): In-Place Dedupe



`nums[fast] == nums[slow] → skip`

slow always marks the end of the clean, deduplicated portion so far

dedupe_in_place

fast/slow

```
slow = 0
for fast in range(1, len(nums)):
    if nums[fast] != nums[slow]:
        slow += 1
        nums[slow] = nums[fast]
return slow + 1
```

Unlike the previous two sections, the two pointers here move in the **same direction**. **fast** advances every step; **slow** only advances when a "new, non-duplicate value" is found, and moves that value into **slow** 's position. **slow** always marks "the end of the clean, deduplicated portion so far." This dedupes **without any extra array**, directly on the original array. The prerequisite is again **a sorted array** — sorting guarantees duplicates sit next to each other, so you only ever need to compare against the last value in the clean portion, not every value that came before.

Use case: in-place dedupe/filter, no extra container allowed for the result

04 Fixed-Size Sliding Window: Slide the Window Instead of Re-Summing

k=3, window slides one step at a time



next step: -4 (leaves), +9 (enters)

O(1) update instead of re-summing k numbers

Only the leftmost element leaves and the rightmost enters — everything in the middle is unchanged

Fixed window — subtract the left edge, add the right edge

fixed window

```
window_sum = 0
max_sum = None
for right in range(len(nums)):
    window_sum += nums[right]           # new element enters the window
    if right >= k - 1:                  # window has accumulated k elements
        if max_sum is None or window_sum > max_sum:
            max_sum = window_sum
        left = right - k + 1
        window_sum -= nums[left]        # the leftmost element leaves the window
return max_sum
```

The brute-force approach re-sums k numbers from every starting point ($O(n \cdot k)$). The key insight of a sliding window: **moving from one window position to the next only drops one element off the left and adds one on the right — everything in between is unchanged**. No need to re-sum; just "add one, subtract one," dropping the per-step cost from $O(k)$ to $O(1)$, and the total to $O(n)$. **right >= k - 1** checks whether the window has accumulated k elements yet (0-indexed, so once you've collected k elements, right's index is exactly k-1). **left = right - k + 1** derives the window's left edge from the right edge, so you don't need a separate left variable.

Use case: sum/extremum over any fixed-length-k contiguous subarray/substring

05 Variable-Size Window (Shrinking): Right Expands, Left Contracts

right expands → window_sum >= target



left contracts (while loop)

keeps shrinking until it no longer qualifies

Outer for expands right; inner while contracts left as far as it still qualifies

```

left = 0
window_sum = 0
min_length = float("inf")
for right in range(len(nums)):
    window_sum += nums[right]           # right keeps expanding the window
    while window_sum >= target:         # as long as the window already qualifies
        current_length = right - left + 1
        min_length = min(min_length, current_length)
        window_sum -= nums[left]       # keep shrinking from the left, trying for shorter
        left += 1

```

Unlike the fixed window, the window size here isn't fixed. `right` keeps expanding rightward (giving the window a "chance" to qualify); the moment it qualifies, the inner `while` keeps shrinking from the left and recording the length, until the window no longer qualifies — only then does control return to the outer loop to keep expanding `right`. **The outer for loop has a while nested inside it, but `left` and `right` together move at most n steps total across the whole run, so the overall complexity is still $O(n)$, not $O(n^2)$ — this is the single most commonly mis-judged complexity in this section.**

Use case: "shortest contiguous subarray satisfying some condition ($\text{sum} \geq \text{target}$)"

06 Variable-Size Window (Dedup): Maintain Window Contents with a Set

longest_valid_window – same skeleton, different validity check

dedup window

```

seen = set()
left = 0
max_length = 0
for right in range(len(text)):
    while text[right] in seen:         # this character is already in the window
        seen.remove(text[left])       # remove from the left one at a time until no conflict
        left += 1
    seen.add(text[right])              # now it's safe to add the new character
    max_length = max(max_length, right - left + 1)
return max_length

```

Almost the identical structure to § 05 (right expands + inner loop contracts) — the difference is **how "is the window valid" gets checked**: § 05 uses a number (`window_sum`), this one uses a `set` to check "does the window already contain this character?" `seen` always exactly mirrors "the characters currently present in window `[left, right]`" — every time `left` moves, the corresponding character must be removed from the set to keep it in sync with the window's actual range.

Use case: "longest contiguous substring satisfying some membership/uniqueness constraint"

07 The Four Patterns, Side by Side

PATTERN	POINTER MOVEMENT	PREREQUISITE	TYPICAL PROBLEMS
Opposite pointers	Both ends toward the middle	Usually needs sorted input	Two-sum, palindrome check
Fast/slow pointers	Same direction, different speeds	Usually needs sorted input	In-place dedupe/filter
Fixed sliding window	Move together, width unchanged	None	Fixed-length subarray sum/extremum
Variable sliding window	Right expands, left contracts	None	Shortest/longest subarray meeting a condition

- All five templates in one place

```
def two_pointer_opposite(items, condition_check):
    left, right = 0, len(items) - 1
    while left < right:
        result = condition_check(items[left], items[right])
        if result == 0:
            return left, right
        elif result < 0:
            left += 1
        else:
            right -= 1
    return None
```

```
def dedupe_in_place(nums):
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

```
def fixed_window(nums, k):
    if k <= 0 or k > len(nums):
        return 0
    window_sum = 0
    best = None
    for right in range(len(nums)):
        window_sum += nums[right]
        if right >= k - 1:
            best = window_sum if best is None else max(best, window_sum)
            window_sum -= nums[right - k + 1]
    return best
```

```
def shrinking_window(nums, target):
    left = 0
    window_sum = 0
    min_length = float("inf")
    for right in range(len(nums)):
        window_sum += nums[right]
        while window_sum >= target:
            min_length = min(min_length, right - left + 1)
            window_sum -= nums[left]
            left += 1
    return min_length if min_length != float("inf") else 0
```

```
def longest_valid_window(items):
    seen = set()
    left = 0
    max_length = 0
    for right in range(len(items)):
        while items[right] in seen:
            seen.remove(items[left])
            left += 1
        seen.add(items[right])
        max_length = max(max_length, right - left + 1)
    return max_length
```

08 Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
Opposite pointers on an unsorted array	"sum too small → move left" only holds when the array is sorted; unsorted, moving a pointer gives no guarantee the sum moves toward target	Sort first (watch out for index requirements), or use a different approach
Fixed window subtracts the left edge before checking <code>right >= k - 1</code>	Before the window has <code>k</code> elements, <code>right - k + 1</code> can be negative — Python then reads from the end of the array, silently wrong	Only subtract inside <code>if right >= k - 1:</code>
Variable window uses <code>if</code> instead of <code>while</code> to shrink	A single shrink may still leave the window valid, missing a shorter answer	Use <code>while window_sum >= target:</code>
Dedup window checks membership with a list	List <code>in</code> is $O(n)$, degrading the whole algorithm back to $O(n^2)$	Use a <code>set</code> , $O(1)$ average membership check
Fast/slow pointers skip the empty-array check	Happens to not crash on empty input (<code>range(1,0)</code> is empty), but relying on that "accidentally correct" behavior is fragile	Explicitly <code>if not nums: return 0</code>
"Nested for + while must be $O(n^2)$ "	Misjudges complexity by pattern-matching loop shape instead of counting actual pointer movement	Left only ever moves forward, total $\leq n$ steps across the whole run → $O(n)$

09 FAQ

Q Are "two pointers" and "sliding window" the same thing?

Think of sliding window as a **special case / application** of two pointers. "Two pointers" broadly means "use two index variables instead of nested loops" (covering opposite, fast/slow, and other movement styles). "Sliding window" specifically means the scenario where two pointers bracket a contiguous range (a window) that changes as the pointers move. Of this chapter's four patterns, the first two (opposite, fast/slow) are other uses of two pointers; the last two (fixed/variable window) are sliding window in the strict sense.

Q When do I use a fixed window vs. a variable window?

Ask whether the problem gives you "a known length, find the extremum at that length" or "an unknown length, find the shortest/longest satisfying some condition." The former (e.g. "max sum over every length-`k` subarray") uses a fixed window; the latter (e.g. "shortest subarray summing to at least target," "longest substring without repeating characters") uses a variable window, where the window size itself is part of what you're solving for.

Q Why does this chapter keep requiring the array to be "already sorted"?

The correctness of opposite pointers and fast/slow pointers both depend on "the relative order of elements reflecting some monotonic property" — after sorting, moving a pointer produces a predictable direction of change (e.g. moving left always increases the sum; duplicate elements are always adjacent). Without sorting, the same pointer-movement logic loses its guarantee about which direction the result changes, so correctness breaks down — which is why problems fitting these techniques usually state or require a sorted array upfront.

Q How much faster is sliding window than brute force, really?

Take the fixed window: brute force re-sums `k` numbers from every starting point, $O(n \times k)$. Sliding window drops the per-step update cost from "re-add `k` numbers" to "subtract one, add one," $O(1)$, bringing the total to $O(n)$. When `k` or `n` is large the gap is dramatic — e.g. `n=100,000`, `k=1,000`: brute force does on the order of 100 million additions, sliding window only about 100,000.

10 Exercises

◆ EXERCISES

1. **Easy Identify the right pointer pattern** for each: (a) sorted array, find an index pair whose difference equals some value; (b) remove all duplicates from a sorted array, return the new length; (c) find the max sum among all length-5 subarrays; (d) find the shortest contiguous subarray summing to at least 100.
Expected: (a) opposite pointers (b) fast/slow pointers (c) fixed window (d) variable window. Tests reasoning backward from the problem description to the pattern, not memorized code.
2. **Medium Fixed window: minimum-sum window** — `nums = [4, 2, 8, 1, 9, 3]`, $k=3$. Return the smallest sum among all length-3 contiguous subarrays.
Expected: **11** (`[2,8,1]=11` is the minimum, checking against `[8,1,9]=18`, `[1,9,3]=13`, `[4,2,8]=14`). Tests whether you can flip the fixed-window template from max to min, or only memorized the max version.
3. **Medium Variable window with negatives** — `nums = [1, -1, 5, -2, 3]`, find the longest contiguous subarray summing to 3. (Hint: with negative numbers, `window_sum >= target` loses its monotonicity, so the plain variable-window template doesn't apply here — this needs prefix sums + a hash table instead.)
Tests recognizing the **boundary of where this chapter's templates apply** — with negative numbers present, window sum is no longer monotonic, and this problem is a preview of the next technique (prefix sum + hash table).
4. **Medium Dedup window variant** — `text = "eceba"`, find the longest substring containing at most two distinct characters.
Expected: **3** ("ece"). Tests converting the "set checks duplication" idea into "a dict counts each character, check whether the distinct count exceeds 2" — a variant on the longest-substring-without-repeats template.
5. **Medium Rewrite from memory, no hints** — close this sheet and independently write `two_sum_sorted`, `is_palindrome`, `max_sum_subarray_k`, and `longest_substring_without_repeat`.
Tests full, independent reproduction of all four movement patterns — the final self-check standard for this chapter.

11 Review Order + Checklist

◆ Must master

Opposite-pointer template: left/right close in, needs sorted input

Fast/slow template: slow marks the clean end, fast keeps probing

Fixed window: `right >= k-1` checks readiness, "subtract left, add right"

Variable shrinking window: while, not if, to fully contract

Variable dedup window: set (not list) for $O(1)$ membership checks

◆ Worth practicing

"Sorted array + relation between two numbers" → think opposite pointers

"In-place dedupe/filter, no extra space" → think fast/slow pointers

"Fixed-length subarray" → think fixed window

"Shortest/longest satisfying a condition" → think variable window

Judge complexity by total pointer movement, not loop nesting shape

◆ Later

Negative numbers break window monotonicity → prefix sum + hash table

Multiple simultaneous windows/pointers (e.g. 3Sum)

Deque-maintained window max/min (monotonic queue)

String-matching problems (minimum window substring) using a dict instead of a set

◆ Chapter checklist

Opposite pointers require sorted input

slow marks the clean deduplicated end

Fixed window checks readiness with `right >= k-1`

Variable window shrinks with while, fully, not partially

Dedup window uses set for $O(1)$ lookup

Overall sliding window is $O(n)$, not $O(n^2)$

Can identify the right pattern from a problem description alone

Once every box is checked, you have four reusable patterns that replace a whole category of brute-force nested loops — and just as importantly, the judgment to recognize which one a new problem calls for before you write a single line. The exercise about negative numbers is a deliberate preview: sliding window has a boundary, and prefix sums + hash tables pick up exactly where it stops working.